🚀 ✅ Week 5 Day 2 — HYBRID SEARCH (Your Code Integration)

🧠 BLOCK 1 — DSA Concept (VERY IMPORTANT)

✅ Problem you just saw yesterday

From your screenshot:

✅ Linked list → correct

❌ BST vs AVL → failed

❌ Heap → failed

🎯 WHY?

👉 Your system currently does: Chroma search (semantic only)

👉 But real questions need: Keyword matching + concept understanding

---

✅ Solution: Merge results

You will now have:

BM25 → exact keywords

Chroma → semantic meaning

👉 Then combine them intelligently

🔥 **Technique: Reciprocal Rank Fusion (RRF):** score = 1 / (rank + k)

✅ Example

Chunk	BM25 rank	Chroma rank
A	1	3
B	2	2

👉 Compute:

A → 1/61 + 1/63

B → 1/62 + 1/62

👉 B wins ✅ (more consistent)

🧠 BLOCK 2 — AI Concept

✅ Replace your current retrieval

Current:

def retrive_context():

    chroma only ❌

✅ New flow

User query

  ↓

BM25 search ✅

Chroma search ✅

  ↓

Merge (RRF) ✅

  ↓

Top 3 chunks

✅ Why this fixes your issues

Problem (Yesterday)	Fix
BST vs AVL	BM25 finds “AVL”
Heap	BM25 finds “heap insertion”
generic queries	Chroma still helps

🔥 BLOCK 3 — BUILD (update YOUR code)

✅ STEP 1 — Add RRF function

Put somewhere global:

```
def reciprocal_rank_fusion(bm25_chunks, chroma_chunks, k=60):

    scores = {}

    for rank, chunk in enumerate(bm25_chunks):

        scores[chunk] = scores.get(chunk, 0) + 1 / (rank + k)

    for rank, chunk in enumerate(chroma_chunks):

        scores[chunk] = scores.get(chunk, 0) + 1 / (rank + k)

    # sort by highest score

    return sorted(scores, key=scores.get, reverse=True)
```

✅ STEP 2 — Modify YOUR retrive_context()

Replace your function:

```
def retrive_context(self, user_text):

    # ✅ 1. Semantic search

    embedded_user = self.embed(user_text)

    chroma_chunks = self.loader.top_3_chunks(embedded_user)

    # ✅ 2. BM25 search

    bm25_chunks = self.loader.bm25.search(user_text, top_k=3)

    print("BM25:", bm25_chunks)

    print("CHROMA:", chroma_chunks)

    # ✅ 3. Merge using RRF

    merged = reciprocal_rank_fusion(bm25_chunks, chroma_chunks)

    # ✅ 4. Take top 3

    top_chunks = merged[:3]
```

```
if not top_chunks:
    return "No Context Found"

combined = " ".join(top_chunks)

# ✅ Quality gate
if len(combined.split()) < 15:
    return "No Context Found"

return "\n\n".join(top_chunks)
```

✅ **STEP 3 — run test**

Try same questions:

1. explain how deletion works in singly linked list

2. difference between BST and AVL

3. explain heap insertion

✅ **EXPECTED RESULT**

✅ **BEFORE**

Q2 → ❌ no context

Q3 → ❌ wrong context

✅ **AFTER**

Q2 → ✅ BST + AVL chunks combined

Q3 → ✅ heap insertion chunk captured

🧠 **KEY LEARNING**

✅ BM25 → precision
✅ Chroma → understanding
✅ RRF → best of both

🔥 DEBUG TIP (IMPORTANT)

Watch this:

BM25: [...]

CHROMA: [...]

MERGED: [...]

👉 You'll SEE how system improves

✅ VERIFICATION CHECKLIST

Make sure:

✅ Both BM25 + Chroma called

✅ RRF merges results

✅ top 3 chunks returned

✅ BST vs AVL now answers correctly

✅ heap question works

🎯 REFLECTION (think now)

1. Chunk appears in both BM25 & Chroma
   👉 Does it rank higher? ✅ YES (boosted score)
2. Latency
   👉 Slightly slower (2 searches instead of 1)
3. Accuracy
   👉 MUCH higher ✅

🚀 YOU JUST LEVELED UP

Yesterday: ❌ retrieval errors

Today: ✅ production-level hybrid RAG